Journal of Future Artificial Intelligence and Technologies

E-ISSN: 3048-3719

Research Article

PIX2DB: Reconstructing Executable Relational Databases from Document Images with a 2B Vision-Language Model

Musa Peker [1,*]

[1] Ozveri R&D Center, Ozdilek Home Textile Industry and Trade Inc., [postal address], Bursa, Türkiye; e-mail: musa.peker@ozdilek.com.tr

* Corresponding Author: Musa Peker (musa.peker@ozdilek.com.tr)

Abstract: Much operational data reaches organizations as images: screenshots of spreadsheets, scanned statements, photographed receipts and charts. Current extraction methods return a cell grid or HTML that keeps the layout but leaves the relational decisions open: merged cells stay merged, a document header and its line items stay unrelated, and formatted numbers remain strings. This study defines image-to-database reconstruction, in which a model must turn one image into a SQLite dump whose tables, column types, primary and foreign keys and rows load and can be queried directly. Training data with exact gold databases is produced by rendering samples of the Spider databases at six layout levels and by converting PubTabNet, FinTabNet and CORD annotations under the same normalization rules. A 2B-parameter model, Qwen3-VL-2B adapted with LoRA on 36,589 image-dump pairs and followed by a deterministic verify-and-repair layer, is compared with Qwen3-VL models of 2B, 4B and 8B parameters and with an HTML-to-SQLite pipeline. On rendered tables from unseen databases it reaches 0.86 table exact match and 0.99 foreign-key recall, against 0.75 and 0.75 for the 8B model prompted for SQL. On held-out real images it obtains cell F1 of 0.81 (PubTabNet) and 0.96 (FinTabNet) and cell recall of 0.93 on receipt photographs. On TabRecSet, a source not used for training, it is the strongest system that writes a database directly (0.52) but trails the 8B HTML pipeline (0.58), mainly in reading photographed text. The results show that the relational conventions are learned reliably by a small model, whereas robust reading of unfamiliar images still benefits from larger encoders.

Keywords: Document understanding; Image-to-SQL; Relational schema reconstruction; Synthetic data; Table extraction; Vision-language models

Received: date
Revised: date
Accepted: date
Published: date
Curr. Ver.: date

Copyright: © 2026 by the authors. Submitted for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY SA) license (https://creativecommons.org/licenses/by-sa/4.0/)

1. Introduction

Tabular data is often available only as a picture. Spreadsheets are shared as screenshots, financial statements arrive as scanned PDFs, purchase records exist as photographs of receipts, and dashboards are circulated as rendered charts. Before such data can be joined, filtered or summed, it has to be turned back into records. This is the research object of the present study: the reconstruction of a relational database from a single document image.

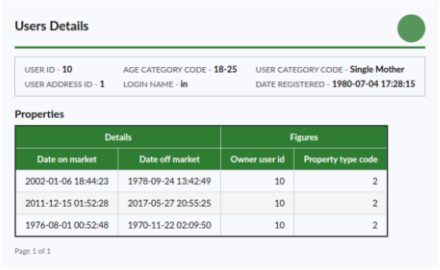
Existing work approaches the problem from the layout side. Table structure recognition (TSR) predicts the grid of a cropped table, from early convolutional detectors [1]–[3] to image-to-sequence and transformer models trained on large corpora [4]–[6]. OCR-free document models parse pages into markup without a separate text recognizer [7]–[11], and chart models translate plots into linearized tables [12]. These methods are strong at describing how a table *looks*: TSR models recover row and column spans accurately, and general vision-language models (VLMs) read printed text well. Their weakness is that the output is still a layout. Turning it into something a program can query requires decisions that markup leaves open: which rows a merged cell belongs to, how a two-level header becomes a column name, whether "(1,209)" is a negative number, and which of two tables on a page refers to the other.

This study addresses the relational step that follows recognition and performs it directly from pixels. Prompting a general model is not enough. In a pilot study with 150 rendered images, Qwen3-VL-8B [13] produced an executable SQL dump for 93% of the images but a completely correct table for only 61%; the same model asked for HTML, converted to SQLite by a short script, reached 46% and failed precisely on merged cells and forms. The errors were rarely misread characters. The models placed merged values in a single row, invented header tables, omitted foreign keys, or wrote a name such as `O'Brien` without escaping, so that one malformed row invalidated an entire statement. The closest prior work, OptiSQL [14], generates one SQL query per question from a clean table image and never reconstructs the database.

We propose PIX2DB, a 2B-parameter model whose output already contains these decisions. Given an image, it writes a SQLite dump with `CREATE TABLE` statements (column types, primary and foreign keys) followed by `INSERT` statements for every record. A deterministic verify-and-repair layer then executes the dump, fixes common syntactic failures and flags suspicious content. Figure 1 shows a rendered document from one of the test databases together with the dump written by the model. The main contributions are:

1. A task definition for image-to-database reconstruction with explicit normalization conventions, and an evaluation protocol that aligns predicted and gold databases without relying on table or column names.
2. A data construction method that yields exact gold databases at scale, by rendering samples of real relational databases at controlled layout difficulty and by converting existing table and receipt annotations into the same conventions, with every gold dump verified by execution.
3. PIX2DB-2b, a LoRA-adapted 2B model with a deterministic repair layer, which outperforms models of the same family with four times more parameters on five of six evaluation sets and recovers foreign keys almost perfectly.
4. Ablations of the training data and the repair layer, and an error analysis on a dataset that was not used for training.

(a) input image



(b) SQLite dump written by pix2db

```
CREATE TABLE users (user_id INTEGER PRIMARY KEY,
  age_category_code TEXT, user_category_code TEXT,
  user_address_id INTEGER, login_name TEXT,
  date_registered TEXT);
CREATE TABLE properties (date_on_market TEXT,
  date_off_market TEXT, owner_user_id INTEGER,
  property_type_code INTEGER,
  FOREIGN KEY (owner_user_id) REFERENCES users(user_id));
INSERT INTO users VALUES
  (10, '18-25', 'Single Mother', 1, 'in', '1980-07-04 17:28:15');
INSERT INTO properties VALUES
  ('2002-01-06 18:44:23', '1978-09-24 13:42:49', 10, 2),
  ('2011-12-15 01:52:28', '2017-05-27 20:55:25', 10, 2),
  ('1976-08-01 00:52:48', '1970-11-22 02:09:50', 10, 2);
```

Figure 1. A rendered document from a test database (a) and the SQLite dump written by PIX2DB (b). The key–value header becomes its own table, the merged *Owner user id* cell is repeated in every item row, and the relation between the two blocks is stated as a foreign key.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 defines the task and describes the data construction, the model, the repair layer and the evaluation protocol. Section 4 reports the experiments and discusses the results. Section 5 compares PIX2DB with the state of the art, and Section 6 concludes.

2. Related Work

2.1. Table Structure Recognition

Table detection and structure recognition have a long history in document analysis, with benchmark competitions such as ICDAR 2013 [15] and layout corpora such as PubLayNet [16]. Deep models moved from detecting rows and columns with convolutional networks [1]–[3] to predicting full structure sequences. PubTabNet [4] introduced 568k scientific tables

with HTML annotations and the tree-edit-distance score TEDS; FinTabNet [17] added tables from financial reports, and PubTables-1M [5] and the corrected FinTabNet.c [18] improved annotation consistency. SciTSR [19] targets complicated scientific tables, while WTW [20] and TabRecSet [21] cover photographed tables in the wild. TableFormer [6] and the PubTables-1M detector predict structure and take the cell text from the PDF or from OCR. Under TEDS, a reproduced rowspan counts as correct, although a database would need the spanned value in every row.

2.2. OCR-Free Document Models and Charts

Layout-aware pre-training such as LayoutLM [22] and LayoutLMv3 [23] combines OCR tokens with visual features. Donut [7] and Pix2Struct [8] showed that an image-to-sequence model can parse documents without an OCR stage, and Nougat [9], GOT [10], olmOCR [11] and Docling [24] convert pages into Markdown, LaTeX or HTML. General VLMs such as Qwen3-VL [13] read tables well enough to be the strongest baselines in this study. For charts, DePlot [12], MatCha [25] and UniChart [26] translate plots into linearized tables, evaluated on ChartQA [27] and PlotQA [28]. None of these models targets a typed relational schema or relations between tables; here a chart is treated as one more layout whose target is a typed table.

2.3. Tables, Images and SQL

Text-to-SQL research assumes that the database already exists. Benchmarks such as Spider [29], BIRD [30] and Spider 2.0 [31] pair questions with databases, and methods improve schema linking [32], constrain decoding to valid SQL [33] or let the model correct its own queries [34]. Visual table question answering [35]–[37] answers questions over images, and recent analyses report that VLMs degrade sharply on noisy real-world tables [38]. OptiSQL [14] renders Spider 2.0 tables and generates an executable query from the image and a question; MTabVQA [36] renders Spider and BIRD tables for multi-table question answering. The present study also renders Spider databases, but the database itself is the output, so that any number of queries can later run on it. Rendering training data with free and exact labels follows a long tradition in text recognition [39].

3. Proposed Method

Figure 2 gives an overview. Training pairs of an image and its gold SQLite dump are built from two sources: rendered samples of real relational databases and converted annotations of real table and receipt images. A 2B VLM is adapted with LoRA to write the dump, and at inference time its output passes through a verify-and-repair layer before it is loaded into SQLite.

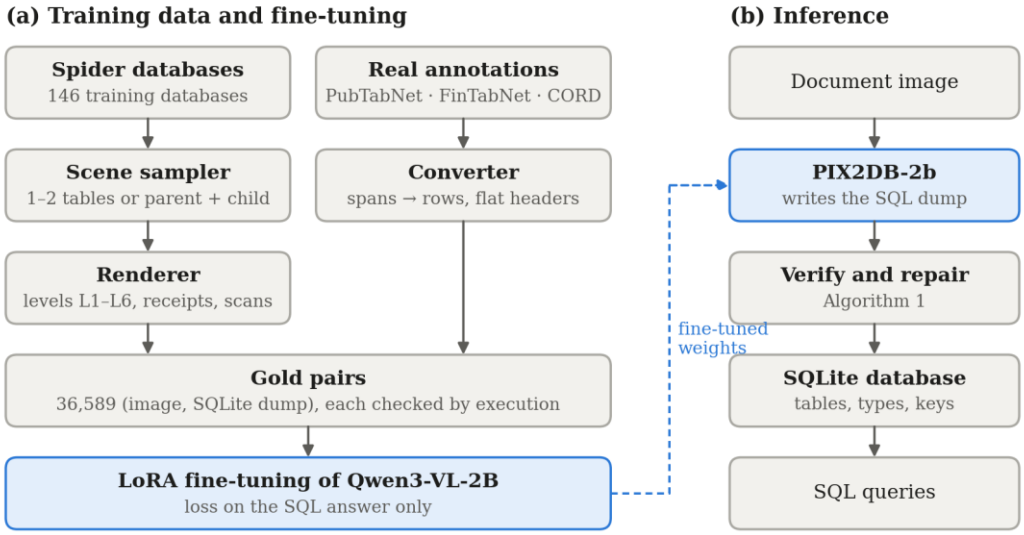


Figure 2. Overview of PIX2DB. (a) Two sources of training pairs and LoRA fine-tuning; (b) inference with the verify-and-repair layer.

3.1. Task Definition

For an image I, the system outputs a SQL text D. The prediction is the database obtained by executing the `CREATE TABLE` and `INSERT` statements of D in an empty SQLite instance; other statements are ignored. The target follows fixed conventions:

- one row per record, with the value of a merged cell repeated in every row it covers;
- multi-level headers joined into one snake_case column name;
- numbers stored without thousand separators or currency signs, accounting negatives such as "(1,209)" as negative numbers, and dates in ISO form;
- empty cells as NULL and derived totals rows not stored;
- one table per logical block, with a foreign key from line items to their header;
- for charts, the category column and one column per data series.

Table and column names are chosen by the model and are not scored, because the same data can be named in many reasonable ways.

3.2. Training Data Construction

3.2.1. Rendering databases

A *scene* is a small database cut from one of the Spider [29] SQLite files (CC BY-SA 4.0): a single table with 3–25 sampled rows and 3–9 columns, two unrelated tables, or a parent row together with 3–15 child rows that reference it through a declared foreign key. Columns with long or mostly empty values are dropped, numeric strings and integer-valued floats are coerced to numbers, and composite primary keys are not declared. Scenes are rendered with HTML and CSS in a headless browser at six levels (Figure 3):

- **L1 clean**: a plain grid with raw column names;
- **L2 styled**: 17 font families, colored or zebra styling, humanized headers in four case styles, thousand separators (never on identifiers or years), currency prefixes on money-like columns and long-form dates;
- **L3 hierarchical**: merged cells on a repeated grouping column, two-level headers that group textual and numeric columns, transposed layouts for small tables and occasional derived totals rows;
- **L4 document**: a parent row rendered as a key–value block above the child table;
- **L5 scan**: L3 and L4 pages degraded with rotation up to ±2.5°, uneven lighting, sensor noise, blur, contrast change, optional grayscale and JPEG quality 45–80;
- **L6 chart**: bar, horizontal bar, line and pie charts with one to three series, printing value labels for 60% of the charts whose labels are exact.

Synthetic receipts use narrow monospaced layouts with three item-line formats, modifiers such as "less sugar", subtotal, tax and payment lines, and are photographed 85% of the time. Two invariants keep the gold exact. Every displayed string maps back to the gold value without loss: formatting adds separators or symbols but never rounds or truncates. Because rendering may reorder rows and columns, for example to group merged cells, the gold scene is reordered in the same way, so the target dump follows the reading order of the image. Splits are made by database: the 146 Spider training databases feed training, the 20 development databases validation, and the 40 databases that appear only in Spider's test set form the synthetic test.

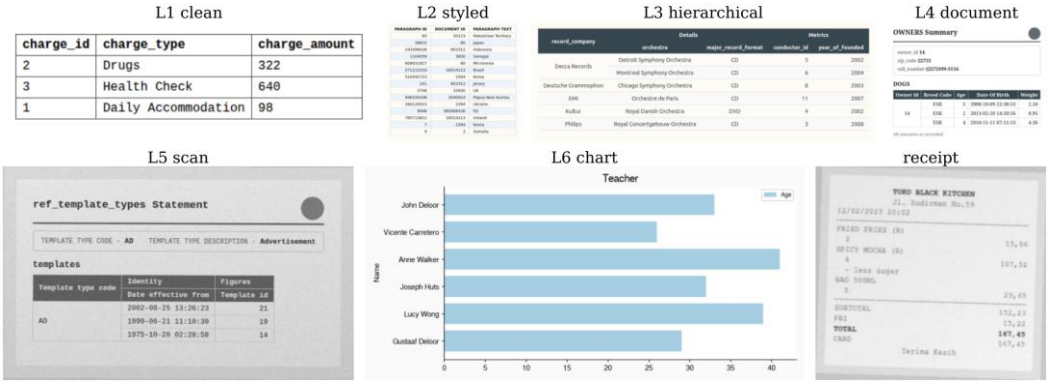


Figure 3. One rendered example per synthetic level, all from the validation databases. For the receipt the gold database is a single *items* table; the subtotal, tax and payment lines are not stored.

3.2.2. Converting real annotations

PubTabNet [4] and FinTabNet [17] (both CDLA-Permissive-1.0) annotate table images with HTML. The converter expands spans into a grid, flattens header rows into names, repeats body rowspans, converts numbers as above and maps lone dashes to NULL. In the HTML release of FinTabNet used here, tables carry no explicit header element, so the first row is taken as the header, and the second as well when the first contains group colspans; the original FinTabNet annotations are used rather than FinTabNet.c [18]. Tables whose body contains colspans, typically section rows such as "Assets", have no unambiguous relational form and are rejected, as are tables with more than 30 rows or 10 columns. Of the streamed training tables, 72% of PubTabNet and 69% of FinTabNet pass. CORD [40] receipts (CC BY 4.0) become an *items* table with name, quantity and price, built from the menu annotations.

3.2.3. Execution check and dataset statistics

Every gold dump, synthetic or converted, is executed and scored against itself; any score below 1.0 indicates a defect in the data. The check found composite keys declared as primary keys, numbers stored as text in Spider, identifiers that start with a digit or are SQL keywords (these are now quoted), empty strings in numeric columns, and one header-deduplication collision in a converted table, which was removed. Table 1 lists all splits; every split is publicly released.

Table 1. Dataset splits. Rows and columns are averages per table; "With FK" is the share of images whose gold database contains a foreign key.

Split	Images	Tables per image	Rows	Columns	With FK
train (synthetic)	24,000	1.38	7.5	3.7	29%
train (real)	18,794	1.00	9.4	4.6	0%
train (receipts)	3,000	1.00	7.1	3.5	0%
validation	600	1.30	7.7	3.7	22%
test (synthetic)	900	1.32	7.0	3.4	24%
test (receipts)	150	1.00	6.8	3.5	0%
PubTabNet (val)	150	1.00	9.4	4.2	0%
FinTabNet (val)	150	1.00	8.5	4.6	0%
CORD (test)	60	1.00	3.5	3.0	0%
TabRecSet (unseen)	150	1.00	6.7	3.8	0%

3.3. Model and Training

PIX2DB-2b is Qwen3-VL-2B-Instruct [13] adapted with low-rank adaptation (LoRA) [41], [42]. For every linear projection W of the language model (attention query, key, value and output, and the MLP gate, up and down projections), the frozen weight is complemented by a trainable low-rank update, as shown in Eq. (1):

$$W' = W + \frac{\alpha}{r} BA \tag{1}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r = 64$ and scaling $a = 128$; B is initialized to zero so that training starts from the base model, and the vision encoder is not adapted. The model receives the image I and a fixed instruction x (Appendix A) and is trained to generate the gold dump y. The loss is the token-level cross-entropy over the answer tokens only, as given in Eq. (2):

$$\mathcal{L}(\theta) = -\frac{1}{|Y|} \sum_{t \in Y} \log p_\theta(y_t \mid y_{<t}, I, x) \tag{2}$$

where Y is the set of positions of the SQL answer; image and instruction tokens and padding are masked out. Training mixes 14,000 synthetic images (all six levels), 3,000 synthetic receipts, 9,999 PubTabNet and 8,000 FinTabNet tables, and 795 CORD receipts seen twice, for a total of 36,589 examples. Images are resized to at most 1.31 MP (about 1,280 visual tokens). Optimization uses AdamW with a learning rate of 10^{-4}, cosine decay, 3% warm-up, an effective batch of 16, bf16 precision and one epoch (2,287 steps, 5.9 h on one NVIDIA A40); further settings are listed in Appendix C. After training, the update of Eq. (1) is merged into the base weights.

3.4. Verify-and-Repair Layer

The model output passes through a deterministic layer before it is returned (Algorithm 1). Statements are executed one at a time. A failing statement is retried after escaping apostrophes inside words, quoting or rewriting identifiers that are SQL keywords or free text (for example `# of sequences`[1] becomes `of_sequences`), and, for `INSERT` statements, inserting row by row with bare composite values such as `9 (8)` or `85.0 ± 10.8` quoted as text and missing trailing cells padded with NULL. A primary key that rejects duplicate values is dropped rather than the rows. Truncated output keeps its complete tuples. Remaining problems are reported rather than hidden: empty tables, values that contradict their declared type, duplicate rows and foreign keys without a parent. Statements that execute are left unchanged, except that a primary key is dropped when later rows violate it. The layer is related in spirit to constrained decoding [33] and self-correction [34] in text-to-SQL, but it operates after generation and needs no additional model call.

Algorithm 1. Verify-and-repair of a generated SQLite dump

Input: model output D (SQL text)
Output: SQLite database S, list of applied repairs R, list of warnings W
1: Split D into statements; if the last statement is incomplete, keep its complete tuples

2: **for each** statement *s* of type CREATE TABLE or INSERT **do**
3: **if** *s* executes in *S* **then continue**
4: try, in order: escape apostrophes; quote keyword identifiers; rename free-text identifiers
5: **if** a variant executes **then** add the repair to *R*; **continue**
6: **if** a UNIQUE constraint fails **then** drop the primary key of that table and retry
7: **if** *s* is INSERT **then** insert row by row, quoting bare values and padding missing trailing cells; drop rows that still fail
8: **if** *s* still fails **then** record the error
9: Check *S*: empty tables, type violations, duplicate rows, dangling foreign keys → *W*
10: **return** *S*, *R*, *W*

3.5. Evaluation Protocol

Cell values are normalized before comparison: Unicode NFKC, unified dash variants, numbers parsed after removing separators and currency signs, common date formats mapped to ISO, and strings case-folded; empty values are ignored. For a gold table *G* and a predicted table *P*, columns are aligned by a maximum-weight bipartite matching whose weights are the multiset overlap of normalized column values, and rows by a second matching whose weights count agreeing aligned cells, both solved with the Hungarian method [43]. The resulting number of agreeing cells $a(G, P)$ is used to match gold tables to predicted tables, again by maximum-weight assignment π. Cell precision and recall are defined in Eq. (3):

$$Prec = \frac{\sum_k a\left(G_k, P_{\pi(k)}\right)}{N_{\text{pred}}}, \quad Rec = \frac{\sum_k a\left(G_k, P_{\pi(k)}\right)}{N_{\text{gold}}} \tag{3}$$

where N_{pred} and N_{gold} are the numbers of non-empty predicted and gold cells. Cell F1 is their harmonic mean, given in Eq. (4):

$$F1 = \frac{2 \cdot Prec \cdot Rec}{Prec + Rec} \tag{4}$$

Table exact match requires a matched table with the same number of rows and columns and every gold cell recovered. *FK recall* is the fraction of gold foreign keys whose child and parent tables are matched to predicted tables connected by a declared foreign key. *Type accuracy* counts an aligned column as correct when its declared SQLite affinity and the stored values agree with the gold column being numeric or textual. Two cases are relaxed. Values of charts drawn without value labels match within the tolerance of Eq. (5):

$$|\hat{v} - v| \le \tau, \quad \tau = 0.04 \cdot \max_i |v_i| \tag{5}$$

and receipts are scored by recall, because a correct database may legitimately contain an extra table for totals or payment.

4. Results and Discussion

4.1. Experimental Setup

All experiments were run on a single NVIDIA A40 (48 GB) with PyTorch 2.8, Hugging Face Transformers and PEFT for training, and vLLM [44] for inference. PIX2DB is compared with Qwen3-VL-2B, 4B and 8B-Instruct given the same instruction (Appendix A), and with a two-stage pipeline in which Qwen3-VL-8B outputs HTML tables that keep row and column spans and a script loads them with `pandas.read_html`, which expands spans, into SQLite. All SQL outputs, those of PIX2DB and of the baselines, go through the same repair layer. Decoding is greedy with at most 3,072 new tokens and the same image budget for all systems.

Besides the synthetic test (150 images per level), held-out splits of the real sources are used: 150 PubTabNet and 150 FinTabNet validation tables and 60 CORD test receipts with at least two items, together with 150 synthetic receipts. These images are unseen, but their style is familiar from training. To measure transfer, TabRecSet-150 was built from TabRecSet [21], [45] (CC BY 4.0), which was not used for training; because it has been public since 2022 it may have been part of the base model's pre-training. Only English original images are taken

(not the generated three-line and no-line variants, which repeat the same tables), one table per image, cropped by its cell polygons and with at least 60% non-empty cells. TabRecSet does not mark header rows, so the first grid row is used as header; 64% of the inspected tables pass the conversion rules. Its annotators wrote illegible characters as #, and these are kept in the gold (94 of 4,199 cells, in 26 tables); removing them from scoring changes cell F1 by less than 0.01 for every system and leaves the ranking unchanged. Some images contain handwriting and three cells contain Chinese characters; neither was filtered. 95% confidence intervals come from 2,000 bootstrap resamples of evaluation items, and *p*-values from a one-sided paired bootstrap against the strongest competitor on each set.

4.2. Main Results

Table 2. Main results with 95% bootstrap intervals. Synthetic: table exact match on 900 images from unseen databases; PubTabNet, FinTabNet and TabRecSet: cell F1; CORD and synthetic receipts (Receipts): cell recall; FK recall on the synthetic test images that contain a foreign key. Best value per column in bold.

System	Synthetic exact	PubTabNet F1	FinTabNet F1	CORD recall	Receipts recall	TabRecSet F1	FK recall
PIX2DB-2b (ours)	**0.86** [0.83, 0.88]	**0.81** [0.77, 0.84]	**0.96** [0.95, 0.98]	**0.93** [0.89, 0.96]	**1.00** [1.00, 1.00]	0.52 [0.47, 0.57]	**0.99**
Qwen3-VL-8B → SQL	0.75 [0.73, 0.78]	0.52 [0.47, 0.57]	0.59 [0.54, 0.64]	0.76 [0.69, 0.83]	0.87 [0.84, 0.90]	0.45 [0.40, 0.50]	0.75
Qwen3-VL-8B → HTML → SQLite	0.47 [0.44, 0.50]	0.78 [0.75, 0.82]	0.87 [0.84, 0.89]	0.44 [0.36, 0.52]	0.38 [0.34, 0.42]	**0.58** [0.53, 0.63]	0.00
Qwen3-VL-4B → SQL	0.67 [0.63, 0.69]	0.48 [0.42, 0.53]	0.48 [0.42, 0.54]	0.79 [0.72, 0.85]	0.77 [0.73, 0.81]	0.38 [0.33, 0.44]	0.25
Qwen3-VL-2B → SQL (base)	0.53 [0.50, 0.56]	0.30 [0.25, 0.36]	0.36 [0.31, 0.41]	0.59 [0.49, 0.68]	0.67 [0.62, 0.72]	0.31 [0.26, 0.36]	0.23

Against the best competitor on each set, PIX2DB is ahead with $p < 0.001$ on synthetic, FinTabNet, CORD and synthetic receipts and with $p = 0.057$ on PubTabNet; on TabRecSet the 8B HTML pipeline is ahead by 0.06 (paired 95% interval of the difference [0.03, 0.09], $p < 0.001$; Wilcoxon signed-rank $p < 0.001$).

Table 2 gives the overall picture. Fine-tuning moves the 2B model from the bottom of the table to the top on every set except TabRecSet. On the synthetic test it gains 33 points of exact match over its own base model and 11 points over the 8B model; on FinTabNet and CORD the margins over the strongest competitor are 9 and 14 points. PubTabNet is the closest case: the HTML pipeline, which suits flat scientific tables, is 3 points behind and the difference is not significant at the 5% level. Foreign keys separate the systems most clearly. PIX2DB declares 0.99 of the gold relations, the 8B model prompted for SQL 0.75, the smaller models about a quarter, and the HTML pipeline none, because nothing in HTML expresses a relation between two tables.

4.3. Effect of Layout Difficulty

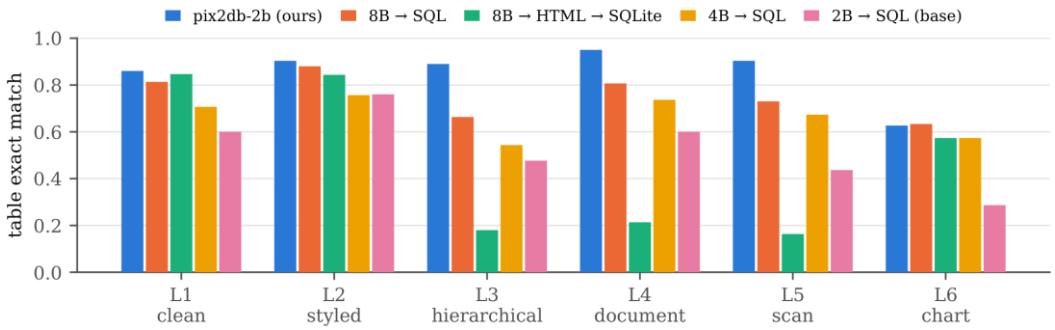


Figure 4. Table exact match per synthetic level (150 images each). The HTML pipeline is competitive on flat tables and loses most of its accuracy once merged cells, forms or scanning are involved.

Figure 4 breaks the synthetic test down by level. On flat tables (L1, L2) all systems are within a few points of each other, and the HTML pipeline reaches 0.85 and 0.84. With merged

cells, key–value headers and photographed pages it falls to 0.18, 0.21 and 0.16, whereas PIX2DB stays at 0.89, 0.95 and 0.90. The HTML stage itself is mostly right; the failure lies in the conversion, which has no notion that a rowspan is a grouping key or that a form above a table is a separate record. Charts are the exception to this pattern: apart from the untuned 2B model, all systems lie between 0.57 and 0.63 exact match, and PIX2DB (0.63) is not better than the 8B model. Charts without value labels are harder (0.52 exact against 0.69 with labels), but labeled charts still fail in almost a third of cases, so the difficulty lies as much in assigning marks to categories and series as in estimating values.

4.4. Contribution of Real Training Data

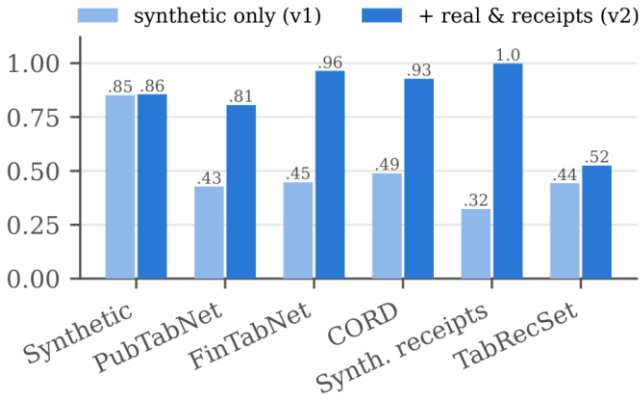


Figure 5. The same model and recipe trained on synthetic data only (v1) or with the converted real tables and receipts added (v2). Metrics as in Table 2.

A first version, otherwise identical, was trained on 24,000 synthetic images only. Figure 5 shows that it reaches the same synthetic accuracy (0.85 against 0.86) but transfers poorly: cell F1 0.43 on PubTabNet, 0.45 on FinTabNet and recall 0.49 on CORD. Its failures on PubTabNet were rarely layout errors. In 82 of 150 outputs the SQL did not execute at all (28 after repair), mostly because composite values such as `9 (8)` or `85.0 ± 10.8` were written as bare numbers, a format that never occurs in Spider; in other cases the model looped on a `CREATE TABLE` statement it could not finish. Adding 18,794 converted real tables and receipts removes these failures without costing synthetic accuracy and lifts TabRecSet from 0.44 to 0.52.

4.5. Effect of the Repair Layer

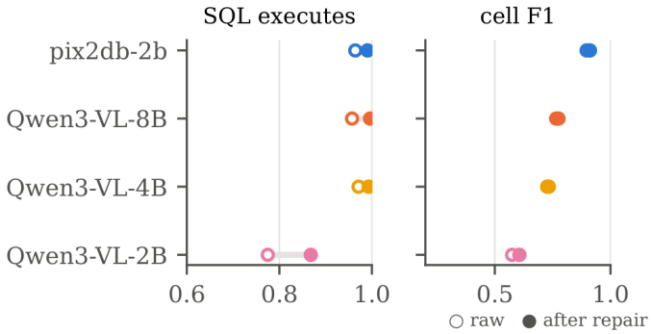


Figure 6. Share of executable outputs and cell F1 before and after the repair layer, pooled over all evaluation sets.

Figure 6 pools all six sets. The layer raises the share of outputs that execute from 0.96 to 0.99 for PIX2DB and from 0.78 to 0.87 for the untuned 2B model, but it moves pooled cell F1 by at most four points. Its value lies in the tail: on TabRecSet, where PIX2DB is out of distribution, execution rises from 0.91 to 0.99, and on PubTabNet F1 improves by 3 points (Appendix B, Table A1). The repair layer is no substitute for training: applied to the base model it leaves F1 at 0.61, far below the fine-tuned model without repair (0.90).

4.6. Error Analysis on an Unseen Source

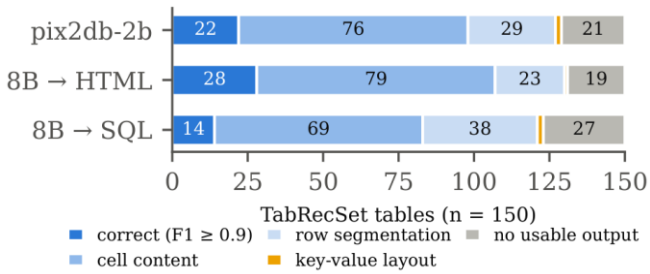


Figure 7. Outcome categories on TabRecSet-150. The thin orange segment is the key–value category (2, 1 and 2 tables from top to bottom); each row sums to 150.

TabRecSet-150 is the only set on which PIX2DB loses (0.52 against 0.58). Each prediction was assigned to one category (Figure 7): F1 of at least 0.9; usable output with the right number of rows but wrong cell contents; a wrong number of rows; a two-column key–value table; or no usable output. Key–value layouts, for which the first-row-as-header gold is questionable, account for only 2 tables and do not explain the gap. The largest bucket for all systems is cell content (76 for PIX2DB, 79 for the HTML pipeline): the rows are found but text in photographed, skewed or low-resolution tables is misread. PIX2DB also segments rows wrongly more often (29 against 23). Both differences concern reading the image rather than organizing its content, and that is where the larger model has the advantage.

4.7. Column Types and Cost

Table 3. Column types and cost on the synthetic test. Type accuracy over aligned columns (after repair), share of raw outputs that execute, mean output tokens and amortized seconds per image with vLLM on one A40.

System	Type accuracy	SQL executes (raw)	Output tokens	Seconds per image
PIX2DB-2b (ours)	0.99	0.96	195	0.09
Qwen3-VL-8B → SQL	0.98	0.95	265	0.29
Qwen3-VL-8B → HTML → SQLite	0.89	–	405	0.34
Qwen3-VL-4B → SQL	0.98	0.99	207	0.18
Qwen3-VL-2B → SQL (base)	0.98	0.83	274	0.13

Table 3 adds column types and output size on the synthetic test. Stored types are almost always right for every SQL system; the HTML pipeline reaches 0.89 because pandas infers types from formatted strings, so "$1,250" stays text. PIX2DB writes the shortest outputs (195 tokens on average, against 405 for HTML), which together with its size makes it the fastest system: 0.09 s per image amortized over a vLLM batch on one A40. Training and every evaluation in this study used about 12 GPU-hours.

4.8. Discussion

Structure and reading. The decisions that turn a layout into a database are few and regular. General VLMs know them but apply them inconsistently; the 8B model declares three quarters of the foreign keys and resolves merged cells correctly about two times in three. After training on a few tens of thousands of exact examples, the 2B model applies them almost every time. Reading behaves differently. The synthetic-only model read rendered text well and real text poorly (Section 4.4), and on TabRecSet the larger general model still reads photographed text better (Section 4.6). Since nothing in the data recipe depends on the size of the base model, training a 4B or 8B variant is the most direct way to close the remaining gap.

Validity of the evaluation. The metric ignores names, so a schema with unhelpful column names scores as well as a good one; naming quality is not measured. The real held-out sets share their sources with the training data and therefore overstate performance on unfamiliar documents, which is why TabRecSet is reported separately. TabRecSet's missing header annotation forces a header assumption that is wrong for some tables, although the error analysis

shows that it affects few. All numbers come from a single training run; the intervals reflect the sampling of test items, not training variance.

Limitations. Handwriting, multi-page tables and tables longer than 30 rows are outside the training distribution. Tables whose body contains section rows were excluded from both training and evaluation, so the behavior of the model on them is unknown. Charts without value labels are read approximately. The data is English, apart from a few TabRecSet cells. Receipts and invoices may contain personal data; the released data uses public datasets whose images were collected and anonymized by their authors. Extraction errors are likely to look plausible, so the warnings reported by the repair layer should be shown to users rather than discarded.

5. Comparison

Table 4 compares the capabilities of PIX2DB with the main families of prior work, as described in the respective papers. Table structure recognition models [5], [6] recover the grid, including spans, but neither type the columns nor resolve spans into records. Page-to-markup models [9], [11], [24] produce HTML or Markdown for whole pages; their output keeps the visual structure and, as the HTML baseline in Section 4 shows, loses accuracy once that structure has to be normalized. Donut-style parsers [7] emit task-specific JSON fields and have been applied to receipts, but they do not produce a general relational schema. Chart-to-table models [12], [26] cover charts only. OptiSQL [14] is closest in its use of SQL, but it generates a query against an existing database for a given question, is evaluated on clean single-table renderings and does not reconstruct the database.

Table 4. Capability comparison with prior work, as described in the respective papers (◐: partly, or for a subset of inputs; –: not applicable). Donut-style parsers emit task-specific JSON fields; OptiSQL emits a query against an existing database.

Method family	Typed schema	Multi-table, FK	Spans resolved	Charts	Photos, receipts	Executable DB
Table structure recognition	✗	✗	✗	✗	◐	✗
Page-to-markup models	✗	✗	✗	✗	◐	✗
Donut-style parsing	◐	✗	✗	✗	✓	✗
Chart-to-table models	✗	✗	–	✓	✗	✗
OptiSQL	–	✗	✗	✗	✗	✗
PIX2DB (ours)	✓	✓	✓	✓	✓	✓

Among the systems evaluated here, the 8B model followed by an HTML conversion script is the strongest baseline for turning a table image into SQLite without task-specific training. Section 4 located its limits: it has no way to express relations between tables, it breaks on merged cells, forms and scans, and it inherits the type inference of the conversion library. Its advantage on TabRecSet shows that a larger encoder still matters for unfamiliar photographs. PIX2DB is the only entry in Table 4 that learns the relational decisions as part of its output, and it does so with a model small enough to process an image in under a tenth of a second on a single GPU.

6. Conclusions

This study defined image-to-database reconstruction and presented PIX2DB, a 2B vision-language model that turns a document image into an executable SQLite database. Its training pairs come with exact gold databases, obtained by rendering parts of the Spider databases and by converting existing table and receipt annotations under one set of conventions. Together with a deterministic verify-and-repair layer, the model outperforms models of its family with up to four times more parameters on five of six evaluation sets, and it is the only system tested that recovers foreign keys reliably.

The experiments separate two abilities. Organizing a table into records, types and relations was learned from a modest amount of exact data, whereas reading text in unfamiliar photographs remained the main source of error. Future work will therefore train larger base models with the same recipe and extend the data to handwriting, multi-page tables, section rows and charts without value labels. The model, data, code and all evaluation outputs are publicly available.

Author Contributions: M.P.: conceptualization, methodology, software, data curation, investigation, formal analysis, visualization and writing (original draft, review and editing).

Funding: This research received no external funding.

Data Availability Statement: The model is available at https://huggingface.co/musapeker/pix2db-2b, the dataset with all training and evaluation splits at https://huggingface.co/datasets/musapeker/pix2db-data, and the code, evaluation outputs and figure scripts at https://github.com/m-peker/pix2db. An interactive demonstration is available at https://huggingface.co/spaces/musapeker/pix2db.

Acknowledgments: A preprint of this manuscript is available on Research Square: https://doi.org/10.21203/rs.3.rs-11467404/v1.

Conflicts of Interest: The author declares no conflict of interest.

References

[1] S. Schreiber, S. Agne, I. Wolf, A. Dengel, and S. Ahmed, "DeepDeSRT: Deep learning for detection and structure recognition of tables in document images," in *Proc. 14th IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, 2017, pp. 1162–1167, doi: 10.1109/ICDAR.2017.192.

[2] S. S. Paliwal, V. D, R. Rahul, M. Sharma, and L. Vig, "TableNet: Deep learning model for end-to-end table detection and tabular data extraction from scanned document images," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2019, pp. 128–133, doi: 10.1109/ICDAR.2019.00029.

[3] D. Prasad, A. Gadpal, K. Kapadni, M. Visave, and K. Sultanpure, "CascadeTabNet: An approach for end to end table detection and structure recognition from image-based documents," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 2439–2447, doi: 10.1109/CVPRW50498.2020.00294.

[4] X. Zhong, E. ShafieiBavani, and A. Jimeno Yepes, "Image-based table recognition: Data, model, and evaluation," in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science, Springer, 2020, pp. 564–580, doi: 10.1007/978-3-030-58589-1_34.

[5] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards comprehensive table extraction from unstructured documents," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 4624–4632, doi: 10.1109/CVPR52688.2022.00459.

[6] A. Nassar, N. Livathinos, M. Lysak, and P. Staar, "TableFormer: Table structure understanding with transformers," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 4604–4613, doi: 10.1109/CVPR52688.2022.00457.

[7] G. Kim *et al.*, "OCR-free document understanding transformer," in *Computer Vision – ECCV 2022*, Lecture Notes in Computer Science, Springer, 2022, pp. 498–517, doi: 10.1007/978-3-031-19815-1_29.

[8] K. Lee *et al.*, "Pix2Struct: Screenshot parsing as pretraining for visual language understanding," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR, 2023.

[9] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," arXiv:2308.13418, 2023, doi: 10.48550/arXiv.2308.13418.

[10] H. Wei *et al.*, "General OCR theory: Towards OCR-2.0 via a unified end-to-end model," arXiv:2409.01704, 2024, doi: 10.48550/arXiv.2409.01704.

[11] J. Poznanski *et al.*, "olmOCR: Unlocking trillions of tokens in PDFs with vision language models," arXiv:2502.18443, 2025, doi: 10.48550/arXiv.2502.18443.

[12] F. Liu *et al.*, "DePlot: One-shot visual language reasoning by plot-to-table translation," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 10381–10399, doi: 10.18653/v1/2023.findings-acl.660.

[13] S. Bai *et al.*, "Qwen3-VL technical report," arXiv:2511.21631, 2025, doi: 10.48550/arXiv.2511.21631.

[14] S. Li, H. Chen, Y. Cai, L. Chen, Q. Ye, and Y. Wang, "OptiSQL: Executable SQL generation from optical tokens," arXiv:2601.13695, 2026, doi: 10.48550/arXiv.2601.13695.

[15] M. Göbel, T. Hassan, E. Oro, and G. Orsi, "ICDAR 2013 table competition," in *Proc. 12th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2013, pp. 1449–1453, doi: 10.1109/ICDAR.2013.292.

[16] X. Zhong, J. Tang, and A. Jimeno Yepes, "PubLayNet: Largest dataset ever for document layout analysis," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2019, pp. 1015–1022, doi: 10.1109/ICDAR.2019.00166.

[17] X. Zheng, D. Burdick, L. Popa, X. Zhong, and N. X. R. Wang, "Global table extractor (GTE): A framework for joint table identification and cell structure recognition using visual context," in *Proc. IEEE Winter Conf. Applications of Computer Vision (WACV)*, 2021, pp. 697–706, doi: 10.1109/WACV48630.2021.00074.

[18] B. Smock, R. Pesala, and R. Abraham, "Aligning benchmark datasets for table structure recognition," in *Document Analysis and Recognition – ICDAR 2023*, Lecture Notes in Computer Science, Springer, 2023, pp. 371–386, doi: 10.1007/978-3-031-41734-4_23.

[19] Z. Chi, H. Huang, H.-D. Xu, H. Yu, W. Yin, and X.-L. Mao, "Complicated table structure recognition," arXiv:1908.04729, 2019, doi: 10.48550/arXiv.1908.04729.

[20] R. Long *et al.*, "Parsing table structures in the wild," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 924–932, doi: 10.1109/ICCV48922.2021.00098.

[21] F. Yang, L. Hu, X. Liu, S. Huang, and Z. Gu, "A large-scale dataset for end-to-end table recognition in the wild," *Scientific Data*, vol. 10, 2023, doi: 10.1038/s41597-023-01985-8.

[22] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2020, pp. 1192–1200, doi: 10.1145/3394486.3403172.

[23] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in *Proc. 30th ACM Int. Conf. Multimedia*, 2022, pp. 4083–4091, doi: 10.1145/3503161.3548112.
[24] C. Auer *et al.*, "Docling technical report," arXiv:2408.09869, 2024, doi: 10.48550/arXiv.2408.09869.
[25] F. Liu *et al.*, "MatCha: Enhancing visual language pretraining with math reasoning and chart derendering," in *Proc. 61st Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 12756–12770, doi: 10.18653/v1/2023.acl-long.714.
[26] A. Masry, P. Kavehzadeh, X. L. Do, E. Hoque, and S. Joty, "UniChart: A universal vision-language pretrained model for chart comprehension and reasoning," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 14662–14684, doi: 10.18653/v1/2023.emnlp-main.906.
[27] A. Masry, D. X. Long, J. Q. Tan, S. Joty, and E. Hoque, "ChartQA: A benchmark for question answering about charts with visual and logical reasoning," in *Findings of the Association for Computational Linguistics: ACL 2022*, 2022, pp. 2263–2279, doi: 10.18653/v1/2022.findings-acl.177.
[28] N. Methani, P. Ganguly, M. M. Khapra, and P. Kumar, "PlotQA: Reasoning over scientific plots," in *Proc. IEEE Winter Conf. Applications of Computer Vision (WACV)*, 2020, pp. 1516–1525, doi: 10.1109/WACV45572.2020.9093523.
[29] T. Yu *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 3911–3921, doi: 10.18653/v1/D18-1425.
[30] J. Li *et al.*, "Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 42330–42357, doi: 10.52202/075280-1835.
[31] F. Lei *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.
[32] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers," in *Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 7567–7578, doi: 10.18653/v1/2020.acl-main.677.
[33] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 9895–9901, doi: 10.18653/v1/2021.emnlp-main.779.
[34] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 36339–36348, doi: 10.52202/075280-1577.
[35] M. Zheng *et al.*, "Multimodal table understanding," in *Proc. 62nd Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 9102–9124, doi: 10.18653/v1/2024.acl-long.493.
[36] A. Singh, C. Biemann, and J. Strich, "MTabVQA: Evaluating multi-tabular reasoning of language models in visual space," arXiv:2506.11684, 2025, doi: 10.48550/arXiv.2506.11684.
[37] M. Mathew, D. Karatzas, and C. V. Jawahar, "DocVQA: A dataset for VQA on document images," in *Proc. IEEE Winter Conf. Applications of Computer Vision (WACV)*, 2021, pp. 2199–2208, doi: 10.1109/WACV48630.2021.00225.
[38] A. Singh, R. Chaudhary, G. Singh, and A. Kumary, "Lost in translation and noise: A deep dive into the failure modes of VLMs on real-world tables," arXiv:2511.17238, 2025, doi: 10.48550/arXiv.2511.17238.
[39] A. Gupta, A. Vedaldi, and A. Zisserman, "Synthetic data for text localisation in natural images," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2315–2324, doi: 10.1109/CVPR.2016.254.
[40] S. Park *et al.*, "CORD: A consolidated receipt dataset for post-OCR parsing," in *Document Intelligence Workshop at NeurIPS*, 2019.
[41] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
[42] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 10088–10115, doi: 10.52202/075280-0441.
[43] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955, doi: 10.1002/nav.3800020109.
[44] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Systems Principles (SOSP)*, 2023, pp. 611–626, doi: 10.1145/3600006.3613165.
[45] F. Yang, L. Hu, X. Liu, S. Huang, and Z. Gu, "TabRecSet: A large scale dataset for end-to-end table recognition in the wild," figshare dataset, 2023, doi: 10.6084/m9.figshare.20647788.

Appendix A. Instruction

All SQL systems receive the image and the following text; PIX2DB was trained with it:

Convert all data shown in this image into a SQLite database. Output only SQL, in a single sql code block: 1. CREATE TABLE statements: snake_case table and column names; column types INTEGER, REAL or TEXT; PRIMARY KEY and FOREIGN KEY constraints where the image implies a relationship (for example a document header and its line items). 2. INSERT statements for every record. Rules: one row per record (repeat the value of a merged cell in every row it covers); flatten multi-level headers into single column names; store numbers without thousand separators or currency symbols; write dates as YYYY-MM-DD; use NULL for empty cells; do not store total or subtotal rows (they are derived); for a chart, create one table with the category column and one column per data series; do not invent rows or columns.

The HTML pipeline instead asks for every table and form in the image as HTML table elements, keeping rowspan and colspan, with each key-value form written as a one-row table, and copying cell text exactly.

Appendix B. Repair Layer per Evaluation Set

Table A1. Share of executable outputs and cell F1 before → after the repair layer. For receipts the F1 column shows cell F1, not the recall used in Table 2.

Set	PIX2DB exec	PIX2DB F1	8B exec	8B F1	2B exec	2B F1
Synthetic	0.96→0.99	0.95→0.97	0.95→1.00	0.94→0.95	0.83→0.91	0.78→0.82
PubTabNet	0.94→0.99	0.78→0.81	0.92→0.98	0.50→0.52	0.52→0.72	0.26→0.30
FinTabNet	1.00→1.00	0.96→0.96	0.99→1.00	0.59→0.59	0.83→0.92	0.33→0.36
CORD	1.00→1.00	0.92→0.92	1.00→1.00	0.54→0.54	0.62→0.68	0.24→0.24
Synth. receipts	1.00→1.00	1.00→1.00	1.00→1.00	0.57→0.57	0.76→0.79	0.34→0.34
TabRecSet	0.91→0.99	0.51→0.52	0.94→0.99	0.42→0.45	0.71→0.87	0.29→0.31

Appendix C. Training and Inference Settings

Table A2. Training and inference settings.

Setting	Value
Base model	Qwen3-VL-2B-Instruct (Apache-2.0)
Adapter	LoRA, $r = 64$, $\alpha = 128$, dropout 0.05; q, k, v, o, gate, up and down projections of the language model
Vision encoder	frozen
Image budget	at most 1,310,720 pixels
Optimization	AdamW, learning rate 10^{-4}, cosine schedule, 3% warm-up, 1 epoch, batch 4 × accumulation 4, bf16, gradient checkpointing
Data	36,589 examples (Table 1); target = gold dump in visual reading order
Hardware and time	1 × NVIDIA A40 (48 GB); 5.9 h
Inference	vLLM, greedy decoding, at most 3,072 new tokens, same image budget
Seed	42 for data generation, sampling and training